

base_sampler

If you want to implement a specific sampling algorithm, you should implement the abstract class :class: BaseSampler .

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

MARK_FIELD module-attribute

```
MARK_FIELD = 'fdz_sample_ismarked'
```

MS_PER_DAY module-attribute

```
MS_PER_DAY = 24 * 60 * 60 * 1000
```

S module-attribute

```
S = TypeVar('S')
```

T module-attribute

```
T = TypeVar('T')
```

TargetType module-attribute

```
TargetType = bool | int | str
```

BaseSampler

```
BaseSampler(seed: int | None = None)
```

Bases: ABC

Base class for sampling implementations.

All subclasses must implement the public :meth: sample method.

Any additional configurations that your custom sampler needs, should be passed during instantiation.

Parameters:

Name	Type	Description	Default
seed	int	Seed for Spark's random number generator.	None

Examples:

```
>>> class RandomSampler(BaseSampler):
...     '''Custom random sampler.'''
...     def __init__(self, fraction: float, seed: Optional[int] = None):
```

```

...     self._fraction = fraction
...     super().__init__(seed=seed)
...     def sample(self, df: DataFrame) -> DataFrame:
...         return df.sample(fraction=self._fraction)
>>> # Initialize the custom sampler
>>> random_sampler_lpc = RandomSampler(0.01)
>>> # Run it against some data
>>> df = ...
>>> random_sampler_lpc.sample(df)

```

TIMESTAMP_MS_FIELD class-attribute instance-attribute [⌕](#)

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

seed property [⌕](#)

```
seed: int
```

sample abstractmethod [⌕](#)

```
sample(df: _TDataFrame) -> _TDataFrame
```

Sample the input data.

Parameters:

Name	Type	Description	Default
df ⌕	<code>_TDataFrame</code>	Input data source.	<i>required</i>

full_recovery [⌕](#)

```
full_recovery(marked_df: _TDataFrame, entity_field: str, recover_column_name: str, mark_field: str = MARK_FIELD)
```

Performs full entity recovery over a sampled DataFrame.

That is, it recovers **all** original instances that belong to any of the sampled entities.

Parameters:

Name	Type	Description	Default
marked_df ⌕	<code>_TDataFrame</code>	Original pyspark DataFrame.	<i>required</i>
entity_field ⌕	<code>str</code>	Name of the entity field.	<i>required</i>
recover_column_name ⌕	<code>str</code>	Name of the column marking instances to be retrieved.	<i>required</i>
mark_field ⌕	<code>str</code>	Name of the field marking sampled instances.	<code>MARK_FIELD</code>

Returns:

Type	Description
DataFrame	A pyspark DataFrame containing all relevant instances.

generate_seed [¶](#)

```
generate_seed() -> int
```

Generate a random seed based on the timestamp in milliseconds.

optimal_recovery [¶](#)

```
optimal_recovery(marked_df: _TDataFrame, entity_field: str, timestamp_field: str, recovery_days: int, recover_column_name: str, mark_field: str = MARK_FIELD) -> _TDataFrame
```

Performs optimal entity recovery over a sampled DataFrame.

That is, for a given sampled instance, it recovers the instances belonging to the same entity and falling in a relevant time period. The relevant period of time is a window before the sampled instance, defined by the recovery_days parameter.

Parameters:

Name	Type	Description	Default
marked_df ¶	_TDataFrame	Original pyspark DataFrame.	<i>required</i>
entity_field ¶	str	Name of the entity field.	<i>required</i>
timestamp_field ¶	str	Name of timestamp field.	<i>required</i>
recovery_days ¶	int	Size of the recovery period. Typically this value should correspond to the maximum window size of your plan's metrics operator.	<i>required</i>
recover_column_name ¶	str	Name of the column marking instances to be retrieved.	<i>required</i>
mark_field ¶	str	Name of the field marking sampled instances.	MARK_FIELD

Returns:

Type	Description
DataFrame	A pyspark DataFrame containing all relevant instances from the original.

recover_entities [¶](#)

```
recover_entities(marked_df: _TDataFrame, entity_fields: dict[str, int | None], mark_field: str = MARK_FIELD, is_optimal_recovery: bool = True, timestamp_field: str | None = None) -> _TDataFrame
```

Performs entity recovery over a sampled DataFrame.

That is, it recovers the relevant original instances that belong to any of the sampled entities.

Parameters:

Name	Type	Description	Default
<code>marked_df</code> ¶	<code>_TDataFrame</code>	Original pyspark DataFrame. It should have a column marking which events were sampled.	<i>required</i>
<code>entity_fields</code> ¶	<code>dict[str, int None]</code>	Dictionary with names and respective recovery period (can be None) for each field to be recovered.	<i>required</i>
<code>mark_field</code> ¶	<code>str</code>	Name of the field marking sampled instances.	MARK_FIELD
<code>is_optimal_recovery</code> ¶	<code>bool</code>	Flags if optimal or full recovery should be performed. If 'True', then <code>timestamp_field</code> and <code>recovery_days</code> are mandatory. Default value is 'True'.	<code>True</code>
<code>timestamp_field</code> ¶	<code>str None</code>	Name of the timestamp field.	<code>None</code>

Returns:

Type	Description
<code>DataFrame</code>	A pyspark DataFrame containing all relevant instances from the original.

`validate_entity_fields` [¶](#)

```
validate_entity_fields(entity_fields: list, recovery_days: int | None) -> tuple[dict[str, int | None], int | None]
```

Validate the types of parameter `entity_fields`.

Returns both a properly formatted dictionary for function `recover_entities`'s '`entity_fields`' parameter input and the corresponding value for `recovery_days`

Parameters:

Name	Type	Description	Default
<code>entity_fields</code> ¶	<code>list</code>	List of field names to be recovered or list of tuples with field names and respective recovery days.	<i>required</i>
<code>recovery_days</code> ¶	<code>int None</code>	Days for recovery. When defined, this will set all entity fields to have the same recovery.	<i>required</i>

Returns:

Type	Description
<code>entity_fields_dict</code>	Dictionary of fields and respective recovery days.
<code>recovery_days</code>	Integer value or None

validate_target_parameters [↗](#)

```
validate_target_parameters(target_field: I, target_value: S) -> tuple[I, set]
```

Validate the types of parameters `target_field` and `target_value`.

Parameters:

Name	Type	Description	Default
<code>target_field</code> ↗	<code>I</code>	Expected to be of str type.	<i>required</i>
<code>target_value</code> ↗	<code>S</code>	Expected to be of a valid type, or a set with all elements belonging to the same valid type. Valid types are: bool, int, str.	<i>required</i>

Returns:

Type	Description
<code>target_field</code>	
<code>target_value</code>	<code>target_value</code> is turned into a set if it wasn't already.

validate_timestamp_field [↗](#)

```
validate_timestamp_field(timestamp_field: str | None, exclude_kickoff_window: bool,
recovery_days: int | None = None) -> None
```

Validate the timestamp field in the context of recovery and kickoff windows.

Parameters:

Name	Type	Description	Default
<code>timestamp_field</code> ↗	<code>str</code>	Name of the timestamp field.	<i>required</i>
<code>exclude_kickoff_window</code> ↗	<code>bool</code>	Whether to exclude the kickoff window.	<i>required</i>
<code>recovery_days</code> ↗	<code>int</code>	Days for recovery. When defined, this will set all entity fields to have the same recovery.	None